

生活服务微服务项目

最终交付材料汇编

整理日期：2026-09-03

范围：用例、需求、概要设计、详细设计、追溯、微服务拆分、接口、数据库归属、跨服务调用、HPA 与技术总结。

最终交付包说明

整理日期：2026-09-03

本目录是最终汇报和提交材料的统一入口。Markdown 文件为可编辑源文件，models/ 下为 Mermaid 模型源文件，PDF 汇编输出到 output/pdf/life-service-delivery-pack.pdf。

阅读顺序

顺序	文件	用途
1	00-交付材料清单.md	对照最终提交清单逐项确认材料位置。
2	01-用例清单-汇报重点.md	查看 UC01-UC21，并确认最终汇报重点讲解用例。
3	02-需求说明书.md	说明业务目标、角色、功能需求、非功能需求和约束。
4	03-概要设计说明书.md	说明总体架构、服务边界、部署、安全和治理设计。
5	04-详细设计说明书.md	说明关键模块、核心流程、接口契约和数据设计。
6	05-需求追溯表.md	追溯需求、用例、代码模块、数据归属和测试报告。
7	06-服务划分图.md 至 09-跨服务调用说明.md	从原整合文档中拆分出的微服务交付材料。
8	10-技术总结报告.md	总结架构演进、工程实践、治理实验和遗留风险。
9	11-自动扩缩容报告-HPA.md	汇总 HPA 配置、压测指标、扩缩容现象和改进建议。

模型源文件

文件	说明
models/service-architecture.mmd	服务划分和数据库归属图。
models/cross-service-checkout.mmd	消费者下单跨服务时序图。
models/traceability.mmd	需求、用例、服务、数据和测试追溯关系图。

关联报告

- 测试报告: reports/testing/microservice-test-report.md、reports/testing/b-side-test-report.md、reports/testing/frontend-e2e-report.md
- 故障处理报告: reports/fault/fault-tolerance-experiment-report-20260903.md
- 压测对比报告: reports/performance/performance-comparison-summary.md
- Sentinel 治理报告: reports/sentinel/sentinel-governance-experiment-report-20260903.md

交付材料清单

整理日期：2026-09-03

本文按最终汇报要求对当前仓库材料做查漏补缺。若某项内容已在原文档中整合，本目录提供拆分后的可提交入口；原始详细材料仍保留在仓库对应目录中。

1. 文档类材料

要求	当前交付入口	状态	说明
用例清单，标出最终汇报重点讲解用例	docs/delivery/01-用例清单-汇报重点.md	已补齐	UC07、UC08、UC11、UC14、UC16、UC18、UC21、UC20 标为重点。
需求说明书	docs/delivery/02-需求说明书.md	已补齐	对齐当前 React + 微服务实现。
概要设计说明书	docs/delivery/03-概要设计说明书.md	已补齐	覆盖总体架构、服务拆分、部署和安全设计。
详细设计说明书	docs/delivery/04-详细设计说明书.md	已补齐	覆盖关键模块、核心流程和接口细节。
追溯表	docs/delivery/05-需求追溯表.md	已补齐	对齐 REQ01-REQ21、UC01-UC21、代码、测试和汇报重点。
技术总结报告	docs/delivery/10-技术总结报告.md	已补齐	总结架构演进、工程化、治理实验和不足。

2. 微服务与治理材料

要求	当前交付入口	状态	说明
测试报告	reports/testing/microservice-test-report.md、 reports/testing/b-side-test-report.md、 reports/testing/frontend-e2e-report.md	已有	微服务 84/84 通过，B 侧专项 63/63 通过；前端本机 E2E 失败原因是浏览器启动权限。
服务划分图	docs/delivery/06-服务划分图.md、 docs/delivery/models/service-architecture.mmd	已拆分	原整合材料来自 微服务划分与数据库归属.md。
服务接口清单	docs/delivery/07-服务接口清单.md	已拆分	原整合材料来自 微服务划分与数据库归属.md 和 frontend/接口说明文档.md。
数据表归属表	docs/delivery/08-数据表归属表.md	已拆分	原整合材料来自 微服务划分与数据库归属.md。
跨服务调用说明	docs/delivery/09-跨服务调用说明.md、 docs/delivery/models/cross-service-checkout.mmd	已拆分	覆盖下单、支付、取消、配送、评价、消息。
自动扩缩容报告 HPA	docs/delivery/11-自动扩缩容报告-HPA.md、 reports/perf/hpa-20260901-102919/、 reports/perf/hpa-20260901-112703/	已补齐	摘要报告和历史 HPA/JMeter 原始归档均保留。

要求	当前交付入口	状态	说明
故障处理报告	reports/fault/fault-tolerance-experiment-report-20260903.md	已有	商家看板依赖订单服务故障降级实验。
压测对比报告	reports/performance/performance-comparison-summary.md	已有	单体版与微服务版性能对比。

3. 工程交付材料

要求	当前文件	状态	说明
README	README.md	已更新	已写清环境版本、端口、启动方法、健康检查、测试账号和初始数据。
Dockerfile	services/*/Dockerfile、frontend/Dockerfile	已有	Gateway、六个业务服务和前端均有 Dockerfile。
流水线配置	.github/workflows/ci.yml	已有	覆盖测试、构建、镜像发布、K8s 部署和诊断归档。
Kubernetes YAML 或 Helm 文件	k8s/*.yaml	已有	当前交付为 K8s YAML，未使用 Helm。
数据库脚本	db/microservices/init-microservice-schemas.sql、db/microservices/ensure-microservice-schemas.sh	已有	多 schema 初始化和非破坏性补齐。
部署脚本	scripts/deploy-kind.sh	已有	CI/CD 和本地 kind 部署入口。
回滚/恢复脚本	scripts/fault-restore-dependency.sh	已有	用于故障实验中恢复依赖服务副本和 HPA。

4. PDF 与模型源文件

类型	输出位置	说明
可编辑文件	docs/delivery/*.md	Markdown 源文件，可直接编辑。
模型源文件	docs/delivery/models/service-architecture.mmd、docs/delivery/models/cross-service-checkout.mmd、docs/delivery/models/traceability.mmd	Mermaid 图源文件，可导入支持 Mermaid 的编辑器。
PDF 汇编	output/pdf/life-service-delivery-pack.pdf	由本目录交付 Markdown 生成。

5. 当前保留和清理口径

- 保留 frontend/参考文档/参考文档/ 作为课程需求和设计来源。

- 保留 2026-09-03 Sentinel、故障容错和性能对比报告作为当前交付依据。
- 2026-09-01 HPA/JMeter 实验材料已复原到 reports/perf/，用于本地无额外备份时继续留档。
- 重复嵌套故障采集目录和被新版替代的旧故障报告仍维持清理，避免交付时误读旧结论。

用例清单与最终汇报重点

整理日期：2026-09-03

标注说明：

- 重点讲解：建议在最终汇报中展开讲流程、接口、服务协作和测试依据。
- 辅助展示：可作为页面或接口覆盖补充说明。

1. 用例总表

UC	场景	汇报级别	前端入口	覆盖接口/服务	状态
UC01	多角色注册、登录与会话建立	辅助展示	/login	/api/captcha、/api/auth/、JWT	已实现
UC02	游客/用户发现商家与商品	辅助展示	/、/search、/merchants/:id	merchant-service 公开目录接口	已实现
UC03	基于位置/评分/销量获得推荐商家	辅助展示	/	/api/recommend	已实现
UC04	消费者维护个人资料和收货地址	辅助展示	/profile	user-service 资料、地址、收藏	已实现
UC05	消费者管理购物车并形成结算清单	重点讲解	/cart、/merchants/:id、/checkout	user-service -> merchant-service 报价校验	已实现
UC06	消费者领取并在订单中使用优惠券	重点讲解	/coupons、/checkout	settlement-service 领券、锁券、 释放、核销	已实现
UC07	消费者提交外卖订单	重点讲解	/checkout	order-service -> user/merchant/s ettlement	已实现
UC08	消费者支付待支付订单	重点讲解	/checkout、/orders	settlement-service -> order-service 支付推进	已实现
UC09	消费者取消未完成订单	辅助展示	/orders	订单取消、库存释放、优惠券释放	已实现
UC10	消费者追踪配送并确认收货	重点讲解	/orders	fulfillment-service -> order-service	已实现
UC11	消费者评价已完成订单	重点讲解	/reviews/:orderId、/orders	engagement-service -> order-service	已实现
UC12	商家注册/启用并进入经营后台	辅助展示	/login、/merchant-center	商家认证和角色路由	已实现
UC13	商家维护店铺资料	辅助展示	/merchant-center	/api/merchant/profile	已实现
UC14	商家管理商品、规格与库存	重点讲解	/merchant-center	/api/merchant/products/、库存事 实源	已实现

UC	场景	汇报级别	前端入口	覆盖接口/服务	状态
UC15	商家处理本店订单	重点讲解	/merchant-center	/api/merchant/orders/	已实现
UC16	骑手接单并完成配送	重点讲解	/rider-center	fulfillment-service -> order-service 并发状态推进	已实现
UC17	骑手维护个人资料	辅助展示	/rider-center	/api/rider/profile	已实现
UC18	管理员管理用户/商家/骑手状态	重点讲解	/admin-center	/api/admin/users、/api/admin/merchants、/api/admin/riders	已实现
UC19	管理员查看平台订单	辅助展示	/admin-center	/api/admin/orders/	已实现
UC20	多角色查看运营看板	重点讲解	/merchant-center、/rider-center、/admin-center	商家看板、骑手看板、管理聚合	已实现
UC21	角色间订单会话沟通	重点讲解	/messages、/messages/orders/:orderId	engagement-service 消息、线程、未读数	已实现

2. 最终汇报建议重点

建议主讲 8 个用例：

- UC07 下单：最能体现微服务拆分后的跨服务校验、库存预占、锁券和订单本地事务。
- UC08 支付：体现结算服务与订单服务之间的状态推进和支付流水。
- UC11 评价：体现互动服务如何校验订单完成、写评价、标记订单明细已评价。
- UC14 商品规格库存：体现商家商品服务是商品和库存事实源。
- UC16 骑手配送：体现履约服务与订单状态机协作。
- UC18 管理员主体管理：体现多角色权限和后台治理能力。
- UC20 商家看板：可串联故障处理报告，展示依赖订单服务故障时的降级能力。
- UC21 订单会话：体现消息参与人校验和多角色协作。

3. 用例覆盖结论

- UC01-UC21 均有前端页面、API 客户端或后端接口覆盖。

- 微服务测试报告覆盖 Gateway、六个业务服务和内部接口，最近报告为 84/84 通过。
- B 侧专项测试覆盖商家、用户、履约、互动边界，最近报告为 63/63 通过。
- Playwright E2E 默认使用 mock API 验证页面流程；真实 Gateway 验收需使用完整运行环境。

软件需求说明书

版本：V2.1

整理日期：2026-09-03

1. 范围

本系统是面向校园生活服务场景的 Web 平台，支持游客浏览，普通用户消费，商家经营，骑手配送和管理员治理。当前实现采用 React 前端、Spring Cloud 微服务、API Gateway、Nacos、MySQL、Redis、Docker Compose 与 Kubernetes 示例部署。

2. 参与者

参与者	说明
游客	未登录访问者，可浏览商家、商品、搜索和公开评价。
普通用户	可注册登录、维护资料地址、加购、领券、下单、支付、确认收货、评价和消息沟通。
商家	可维护店铺、商品、规格、库存、查看看板并处理本店订单。
骑手	可维护资料、查看任务、接单、取餐、送达和查看配送统计。
管理员	可管理用户、商家、骑手和订单，执行审核、冻结和解冻。
外部基础设施	MySQL、Redis、Nacos、Kubernetes、对象存储或本地文件存储。

3. 功能需求

需求编号	需求名称	对应用例	优先级	当前状态
REQ01	多角色注册登录和会话建立	UC01	高	已实现
REQ02	商家与商品浏览搜索	UC02	高	已实现
REQ03	推荐商家	UC03	中	已实现
REQ04	用户资料、地址和收藏管理	UC04	高	已实现
REQ05	购物车管理	UC05	高	已实现
REQ06	优惠券领取、锁定、释放和核销	UC06	高	已实现

需求编号	需求名称	对应用例	优先级	当前状态
REQ07	提交外卖订单	UC07	高	已实现
REQ08	模拟支付和支付查询	UC08	高	已实现
REQ09	取消订单并释放资源	UC09	高	已实现
REQ10	配送追踪和确认收货	UC10	高	已实现
REQ11	已完成订单评价和图片上传	UC11	中	已实现
REQ12	商家注册登录	UC12	高	已实现
REQ13	商家店铺资料维护	UC13	高	已实现
REQ14	商家商品、规格和库存管理	UC14	高	已实现
REQ15	商家处理本店订单	UC15	高	已实现
REQ16	骑手接单和配送完成	UC16	高	已实现
REQ17	骑手资料维护	UC17	中	已实现
REQ18	管理员主体审核、冻结和解冻	UC18	高	已实现
REQ19	管理员查看平台订单	UC19	中	已实现
REQ20	多角色运营看板	UC20	中	已实现
REQ21	订单会话消息	UC21	中	已实现

4. 非功能需求

类别	需求
可运行性	支持 Docker Compose 本地启动，Kubernetes YAML 示例部署。
可维护性	前端按页面组件拆分，后端按领域服务拆分。
可测试性	提供后端全量测试、B 侧专项测试、前端单测和 Playwright E2E。
安全性	JWT 鉴权，服务侧角色校验，冻结状态运行时拦截。
数据隔离	一个业务表只有一个 Owner 服务，其他服务通过内部接口访问快照。
可观测性	服务暴露 /api/health，Gateway 暴露 /actuator/health，部署流水线归档诊断。
弹性	K8s HPA 可按 CPU 指标扩缩容，核心依赖链路有故障降级实验。

5. 边界说明

- 支付为模拟支付，不接第三方支付网关。
- 骑手调度为演示级任务流转，不含地图和复杂派单算法。
- 消息已持久化为订单会话，不是实时 WebSocket 聊天。
- 图片上传支持本地/OSS 配置，生产级 CDN、防盗链、生命周期策略为后续增强。
- Playwright E2E 默认使用 mock API，不能替代真实 Gateway 冒烟。

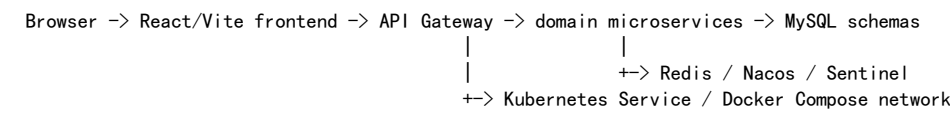
软件概要设计说明书

版本：V2.1

整理日期：2026-09-03

1. 总体架构

系统采用前后端分离和微服务架构：



前端负责页面交互、路由保护和 API 调用；Gateway 负责统一入口和路由；业务服务按领域拥有各自数据库；跨服务访问通过 OpenFeign 和内部接口完成。

2. 服务划分

服务	职责	数据库
api-gateway	/api 路由、路由优先级、请求头透传	无业务库
common-lib	JWT、Result、异常、健康检查、Feign 常量	无业务库
merchant-service	商家、商品、分类、搜索、推荐、商家后台、管理员商家管理	merchant_db
user-service	消费者、管理员、资料、地址、购物车、收藏、管理员用户管理	user_db
order-service	下单、订单状态、商家订单、管理员订单、履约内部接口、补偿记录	order_db
settlement-service	优惠券、用户券、锁券、核销、支付流水、模拟支付	settlement_db
fulfillment-service	骑手、配送任务、骑手看板、管理员骑手管理	fulfillment_db
engagement-service	评价、图片上传、商家评分、消息线程和未读数	engagement_db

3. 前端概要设计

模块	文件	职责
应用入口	frontend/src/main.jsx	React 装配、路由、导航、受保护路由。
API 客户端	frontend/src/utis/api.js	Axios 请求、token 注入、响应展开、错误转换。
会话管理	frontend/src/utis/ApiProvider.jsx	登录态、当前角色、登出、通知。
页面组件	frontend/src/pages/*.jsx	首页、搜索、购物车、订单、商家、骑手、管理等页面。
E2E 支撑	frontend/e2e/support/mockApi.js	Playwright 场景 mock API。

4. 数据设计原则

- 一个业务表只归属一个服务。
- 其他服务只保存必要 ID、金额、名称、图片等业务快照。
- 下单时复制地址、商品、规格、金额快照，避免历史订单随源数据变化。
- 跨服务一致性优先使用同步校验、本地事务、幂等请求和补偿记录。

5. 部署概要

- 本地: docker-compose.yml 启动 MySQL、Redis、Nacos、Gateway、业务服务和前端。
- K8s: k8s/*.yaml 提供 ConfigMap、Secret、MySQL、Redis、Nacos、Gateway、业务服务、前端和 HPA。
- CI/CD: .github/workflows/ci.yml 覆盖测试、前端构建、镜像构建、kind/K8s 部署和诊断归档。

6. 安全与治理概要

- JWT 由认证服务签发，业务服务通过公共拦截器解析身份。
- Gateway 保留 Authorization 请求头，不直接拥有业务鉴权逻辑。
- Nacos 承接服务注册和配置中心。
- Sentinel 承接入口限流、业务服务限流和 Feign 熔断规则。
- Feign 设置基础超时和日志等级，避免跨服务调用无限等待。

7. 质量保障

- 微服务测试: reports/testing/microservice-test-report.md, 84/84 通过。
- B 侧专项测试: reports/testing/b-side-test-report.md, 63/63 通过。
- 性能对比: reports/performance/performance-comparison-summary.md。
- HPA 报告: docs/delivery/11-自动扩缩容报告-HPA.md。
- 故障容错: reports/fault/fault-tolerance-experiment-report-20260903.md。

软件详细设计说明书

版本：V2.1

整理日期：2026-09-03

1. 认证与会话设计

- 消费者和管理员认证由 `user-service` 负责。
- 商家认证由 `merchant-service` 负责。
- 骑手认证由 `fulfillment-service` 负责。
- 登录成功后返回 JWT 和角色信息。
- 前端 `ApiProvider.jsx` 保存 token, `api.js` 自动注入 `Authorization: Bearer <token>`。
- 业务服务通过 `common-lib` 中的拦截器解析用户 ID、角色和主体状态。

2. 下单详细流程

- 前端在 `/checkout` 提交商家、地址、商品、规格、数量和优惠券。
- `order-service` 调用 `user-service` 校验地址归属并获取地址快照。
- `order-service` 调用 `merchant-service` 校验商家、商品、规格、价格和库存。
- `merchant-service` 使用 `requestId` 幂等预占库存。
- `order-service` 调用 `settlement-service` 锁定优惠券。
- `order-service` 本地事务写入 `orders`、`order_item`、必要的 `group_coupon` 和补偿状态。
- 下单成功后调用 `user-service` 清理对应商家购物车。
- 若优惠券锁定或本地写入失败，订单服务释放库存并记录补偿。

3. 支付详细流程

- 前端调用 `POST /api/orders/{id}/pay`。
- Gateway 高优先级路由到 `settlement-service`。
- `settlement-service` 校验订单金额和用户归属。
- 支付服务写入 `payment` 成功流水。
- 调用 `order-service /internal/orders/{id}/mark-paid` 推进订单到待接单状态。
- 若订单状态或金额不匹配，拒绝支付或保留可复核流水。

4. 取消和补偿设计

- 消费者取消订单时，`order-service` 先把订单状态改为 `cancelled`。
- 随后同步调用 `merchant-service` 释放库存、`settlement-service` 释放用户券。
- 任一释放失败时写入 `order_compensation`，订单取消状态仍对用户可见。
- 该设计保证取消动作不因外部依赖短时失败而整体回滚。

5. 骑手履约设计

- `fulfillment-service` 提供骑手看板、任务列表和任务状态更新接口。
- 可接任务、已分配任务和已完成任务由 `order-service` 内部接口提供。
- 接单通过 `order-service` 状态条件更新保证并发安全。
- 送达完成后订单状态推进为完成，消费者侧可确认和评价。

6. 评价和消息设计

- 评价由 `engagement-service` 写入 `review` 表。

- 提交评价前调用 `order-service` 校验订单完成、用户归属和商品明细。
- 评价成功后调用 `order-service` 标记订单明细已评价。
- 图片上传使用 `multipart/form-data`，支持本地存储或 OSS 配置。
- 消息必须绑定订单，发送前校验发送方和接收方都是订单参与者。
- 线程和未读数由 `message` 表聚合得到。

7. 商家看板和故障降级

- `merchant-service` 提供 `GET /api/merchant/dashboard`。
- 正常情况下通过 `order-service /internal/orders/merchant-dashboard` 获取订单统计和收入趋势。
- 当订单服务不可用时，商家服务返回 `degraded=true` 的临时看板数据。
- 故障处理实验见 `reports/fault/fault-tolerance-experiment-report-20260903.md`。

8. 接口错误处理

- 后端业务异常统一包装为 `Result<T>`，包括 `code`、`message`、`data`。
- 分页响应使用 `PageResult<T>`。
- 前端 `api.js` 将非 200 业务码转换为 `ApiError`。
- 401 会触发统一未授权处理，提示重新登录。

9. 测试设计

测试层级	文件/命令	目标
后端全量	<code>scripts/test-microservices.ps1</code>	Gateway 和六个业务服务测试。
B 侧专项	<code>scripts/test-b-side-services.ps1</code>	服务边界、公共包复制、跨服务引用和业务测试。
前端单测	<code>npm.cmd run test:ci</code>	页面和 API 调用契约。
前端 E2E	<code>npm.cmd run e2e</code>	UC01-UC21 页面流程，默认 mock API。

测试层级	文件/命令	目标
Gateway 冒烟	scripts/e2e-b-side-api.ps1	真实 Gateway 环境接口冒烟。

需求追溯表

整理日期：2026-09-03

1. 追溯关系

REQ -> UC -> Frontend Page/API Client -> Gateway Route -> Microservice Controller/Service -> Database Owner -> Test Report

2. 追溯矩阵

需求	用例	汇报级别	前端模块	后端服务	数据归属	测试依据
REQ01	UC01	辅助展示	Login.jsx、ApiProvider.jsx	user-service、merchant-service、fulfillment-service	user_db、merchant_db、fulfillment_db	微服务测试、E2E
REQ02	UC02	辅助展示	Home.jsx、Search.jsx、MerchantDetail.jsx	merchant-service	merchant_db	微服务测试、前端单测
REQ03	UC03	辅助展示	Home.jsx	merchant-service	merchant_db	微服务测试
REQ04	UC04	辅助展示	Profile.jsx	user-service	user_db	微服务测试、前端单测
REQ05	UC05	重点讲解	Cart.jsx、Checkout.jsx	user-service、merchant-service	user_db、merchant_db	微服务测试、B 侧测试
REQ06	UC06	重点讲解	Coupons.jsx、Checkout.jsx	settlement-service、order-service	settlement_db、order_db	微服务测试
REQ07	UC07	重点讲解	Checkout.jsx	order-service、user-service、merchant-service、settlement-service	order_db	微服务测试、性能对比
REQ08	UC08	重点讲解	Orders.jsx、Checkout.jsx	settlement-service、order-service	settlement_db、order_db	微服务测试
REQ09	UC09	辅助展示	Orders.jsx	order-service、merchant-service、settlement-service	order_db	微服务测试
REQ10	UC10	重点讲解	Orders.jsx、RiderConsole.jsx	fulfillment-service、order-service	fulfillment_db、order_db	微服务测试
REQ11	UC11	重点讲解	Review.jsx、ReviewImageGallery.jsx	engagement-service、order-service	engagement_db、order_db	微服务测试、B 侧测试
REQ12	UC12	辅助展示	Login.jsx、MerchantConsole.jsx	merchant-service	merchant_db	微服务测试
REQ13	UC13	辅助展示	MerchantConsole.jsx	merchant-service	merchant_db	微服务测试、前端单测

需求	用例	汇报级别	前端模块	后端服务	数据归属	测试依据
REQ14	UC14	重点讲解	MerchantConsole.jsx	merchant-service	merchant_db	微服务测试、B 侧测试
REQ15	UC15	重点讲解	MerchantConsole.jsx	order-service	order_db	微服务测试
REQ16	UC16	重点讲解	RiderConsole.jsx	fulfillment-service、 order-service	fulfillment_db、order_db	微服务测试、B 侧测试
REQ17	UC17	辅助展示	RiderConsole.jsx	fulfillment-service	fulfillment_db	微服务测试
REQ18	UC18	重点讲解	AdminConsole.jsx	user-service、merchant-serv ice、fulfillment-service	user_db、merchant_db、 fulfillment_db	微服务测试、B 侧测试
REQ19	UC19	辅助展示	AdminConsole.jsx	order-service	order_db	微服务测试
REQ20	UC20	重点讲解	MerchantConsole.jsx、 RiderConsole.jsx、 AdminConsole.jsx	merchant-service、 fulfillment-service、 order-service	多库聚合	微服务测试、故障报告
REQ21	UC21	重点讲解	Messages.jsx、MessageOrderD etail.jsx	engagement-service、 order-service	engagement_db	微服务测试、B 侧测试

3. 测试结果摘要

测试集	报告	结论
微服务全量测试	reports/testing/microservice-test-report.md	84/84 通过。
B 侧专项测试	reports/testing/b-side-test-report.md	63/63 通过。
前端 E2E 本地报告	reports/testing/frontend-e2e-report.md	本机浏览器启动权限 spawn EPERM，不视为业务失败。
性能对比	reports/performance/performance-comparison-summary.md	微服务获得独立部署和治理能力，部分接口延迟更低，整体资源占用更高。
故障容错	reports/fault/fault-tolerance-experiment-report-20260903.md	商家看板依赖订单服务故障时可降级返回。

服务划分图

1. 图源文件

模型源文件: docs/delivery/models/service-architecture.mmd

2. Mermaid 图

```
graph LR
    FE[React 前端] --> GW[API Gateway]
    GW --> US[user-service\n用户/地址/购物车/收藏]
    GW --> MS[merchant-service\n商家/商品/搜索/推荐/看板]
    GW --> OS[order-service\n订单/商家订单/履约代理]
    GW --> SS[settlement-service\n优惠券/支付]
    GW --> FS[fulfillment-service\n骑手/配送]
    GW --> ES[engagement-service\n评价/图片/消息]
    US --> UDB[(user_db)]
    MS --> MDB[(merchant_db)]
    OS --> ODB[(order_db)]
    SS --> SDB[(settlement_db)]
    FS --> FDB[(fulfillment_db)]
    ES --> EDB[(engagement_db)]
    OS --> MS
    OS --> SS
    SS --> OS
    FS --> OS
    ES --> OS
    ES --> US
    ES --> MS
    US --> MS
```

3. 汇报说明

该图用于说明当前微服务拆分后的运行态：前端不直接访问业务服务，所有请求经 Gateway 路由；每个业务服务拥有自己的数据库 schema；跨服务调用通过内部接口完成。

服务接口清单

1. 公开接口

服务	接口范围	说明
api-gateway	/api/、/actuator/health	统一入口和健康检查。
user-service	/api/auth/register、/api/auth/login、/api/auth/admin/login、 /api/user/、/api/admin/users/	消费者、管理员、资料、地址、购物车、收藏、用户管理。
merchant-service	/api/auth/merchant/、/api/merchants/、/api/products/、 /api/categories、/api/search、/api/recommend、/api/merchant/、 /api/admin/merchants/	商家、商品、搜索、推荐、商家后台、管理员商家管理。
order-service	/api/checkout、/api/orders/、/api/merchant/orders/、 /api/admin/orders/	订单主链路、商家订单、管理员订单。
settlement-service	/api/coupons/、/api/orders/{id}/pay、/api/orders/{id}/payments、 /api/payments/{id}	优惠券和支付。
fulfillment-service	/api/auth/rider/、/api/rider/、/api/delivery/、/api/admin/riders/	骑手和配送。
engagement-service	/api/reviews/、/api/uploads/images、/api/products/{id}/reviews、 /api/merchants/{id}/reviews、/api/user/reviews、/api/messages/	评价、上传、评分、消息。

2. 内部接口

提供方	内部接口	调用方	用途
merchant-service	/internal/merchants/{id}、/internal/products/{id}、 /internal/products/quote、/internal/products/reserve、 /internal/products/release	用户、订单、履约、互动	商家商品快照、报价、库存预占和释放。
user-service	/internal/users/{id}、/internal/users/{id}/addresses/{addressId}、 /internal/users/{id}/cart	订单、互动	用户和地址快照、下单后清购物车。
order-service	/internal/orders/{id}、/internal/orders/{id}/mark-paid、 /internal/orders/{id}/participants、/internal/orders/{id}/reviewed-items、 /internal/fulfillment/	结算、履约、互动、商家	订单状态、支付推进、参与人校验、评价标记、骑手任务代理。
settlement-service	/internal/coupon-locks/、/internal/payments/mock-success	订单、联调	锁券、释放、核销、模拟支付。
fulfillment-service	/internal/riders/{id}	订单、互动	骑手快照和状态校验。

3. 更详细接口文档

完整路径、角色和状态见 `frontend/接口说明文档.md`。

数据表归属表

1. 归属原则

- 一个业务表只有一个 Owner 服务。
- 跨服务只通过内部接口获取快照，不直接查表或跨库联表。
- 订单保存下单时的商品、地址、金额快照，保证历史订单稳定。

2. 表归属

表	Owner 服务	Schema	说明
admin	user-service	user_db	管理员账号和状态。
user	user-service	user_db	消费者账号、资料、状态。
address	user-service	user_db	收货地址。
cart	user-service	user_db	购物车和商品展示快照。
user_favorite_merchant	user-service	user_db	用户收藏商家。
merchant	merchant-service	merchant_db	商家账号、门店资料、评分、销量。
category	merchant-service	merchant_db	商品分类。
product	merchant-service	merchant_db	商品主数据。
spec_group	merchant-service	merchant_db	商品规格组。
product_spec	merchant-service	merchant_db	规格价格和库存。
merchant_stock_change	merchant-service	merchant_db	库存预占/释放等记录。
orders	order-service	order_db	订单主表、状态、金额、参与方。
order_item	order-service	order_db	订单明细快照。
group_coupon	order-service	order_db	团购券码预留和履约交付物。
order_compensation	order-service	order_db	跨服务补偿记录。
coupon	settlement-service	settlement_db	优惠券模板。
user_coupon	settlement-service	settlement_db	用户领券、锁定、核销状态。
payment	settlement-service	settlement_db	支付流水。

表	Owner 服务	Schema	说明
rider	fulfillment-service	fulfillment_db	骑手资料、审核和状态。
review	engagement-service	engagement_db	评价内容、评分、图片。
message	engagement-service	engagement_db	订单会话消息和已读状态。

3. 数据库脚本

- 主初始化脚本: db/microservices/init-microservice-schemas.sql
- 非破坏性补齐脚本: db/microservices/ensure-microservice-schemas.sh

跨服务调用说明

1. 调用原则

- 对外统一走 API Gateway。
- 服务间调用使用 OpenFeign + Nacos 服务名。
- 不跨库联表，不引用其他服务 Mapper 或 Service。
- 强一致主流程使用同步调用、幂等键和本地补偿记录。

2. 核心调用链

场景	调用链	失败处理
加入购物车	user-service -> merchant-service /internal/products/quote	报价失败不写购物车，前端提示稍后重试。
下单	order-service -> user-service 地址校验; order-service -> merchant-service 报价和库存预占; order-service -> settlement-service 锁券	库存失败直接终止；锁券失败释放库存；本地写入失败记录补偿。
支付	settlement-service -> order-service /internal/orders/{id}/mark-paid	金额或状态不匹配拒绝；调用异常可按流水和订单状态重试。
取消订单	order-service -> merchant-service 释放库存; order-service -> settlement-service 释放优惠券	释放失败写 order_compensation，取消状态仍保持。
骑手接单	fulfillment-service -> order-service /internal/fulfillment/orders/{id}/assign-rider	订单状态条件更新，接单冲突返回业务错误。
送达完成	fulfillment-service -> order-service /internal/fulfillment/orders/{id}/delivered	失败不写本地错误状态，允许骑手重试。
提交评价	engagement-service -> order-service 校验订单和商品；成功后标记已评价	校验失败不写评价；标记失败保留日志型扩展点。
订单消息	engagement-service -> order-service 校验参与人，并按目标类型查询用户/商家/骑手快照	任一校验服务不可用则不写消息。
商家看板	merchant-service -> order-service /internal/orders/merchant-dashboard	订单服务不可用时返回 degraded=true 临时看板。

3. 图源文件

下单调用链模型源文件: docs/delivery/models/cross-service-checkout.mmd

技术总结报告

整理日期：2026-09-03

1. 项目成果

项目已从早期前端原型和单体后端演进为微服务阶段性交付版本，具备 React 前端、API Gateway、六个业务微服务、多 schema 数据库、Docker Compose、Kubernetes YAML、CI/CD、测试报告和治理实验材料。

2. 技术选型总结

层级	技术	作用
前端	React 18、React Router、Vite、Axios	页面、路由、构建和接口调用。
后端	Java 21、Spring Boot 3.4、MyBatis-Plus	微服务业务开发和持久化。
网关	Spring Cloud Gateway	统一入口和路由。
服务发现/配置	Nacos Discovery / Config	服务注册和配置中心。
服务调用	OpenFeign、Spring Cloud LoadBalancer	跨服务同步调用。
治理	Sentinel、Feign 超时和日志	限流、熔断、降级和调用控制。
数据	MySQL 8、Redis 7	多 schema 业务数据和缓存。
部署	Docker Compose、Kubernetes YAML	本地运行和云端示例部署。
CI/CD	GitHub Actions	测试、构建、镜像发布和部署诊断。

3. 架构收获

- 服务拆分后，每个业务域拥有独立数据库和构建部署边界。
- 订单链路通过内部接口完成地址校验、库存预占、优惠券锁定、支付推进和配送状态更新。
- 商家看板链路通过降级返回避免依赖订单服务故障时整体 500。
- HPA 实验证明 Gateway 和核心业务服务能在 CPU 压力上升时扩容，在压力下降后缩容。

- 性能对比表明微服务带来部署和治理收益，但单机资源占用高于单体。

4. 测试与质量

- 微服务全量测试 84/84 通过。
- B 侧专项测试 63/63 通过。
- 前端单测覆盖主要页面和 API 契约。
- Playwright E2E 覆盖 UC01-UC21，但默认 mock API。
- CI/CD 提供自动化测试、构建、部署和诊断归档。

5. 不足与后续改进

- 支付仍为模拟支付，后续可接真实支付网关和回调验签。
- 骑手调度仍偏简单，后续可接地图、距离和容量约束。
- 消息不是实时通讯，后续可加入 WebSocket 或推送服务。
- 搜索接口在 HPA 压测中表现为主要瓶颈，后续应减少 N+1 查询并分页控制响应体。
- 真实 Gateway 多角色端到端冒烟脚本仍可继续增强。
- 跨服务故障实验目前重点覆盖商家看板链路，后续可扩展到下单、支付、配送和评价。

6. 最终结论

当前项目已经满足课程阶段交付所需的功能、文档、测试、部署和治理材料要求。最终汇报建议围绕“消费者下单支付主链路、商家商品库存、骑手履约、管理员治理、商家看板降级、HPA 自动扩缩容、单体/微服务压测对比”展开。

Kubernetes HPA 自动扩缩容报告

整理日期：2026-09-03

1. 实验目的

验证 Kubernetes HorizontalPodAutoscaler 能否在业务访问压力升高时自动增加 Pod 副本数，并在压力下降后按缩容稳定窗口回落。同时记录吞吐量、响应时间、P95、错误率和资源变化。

2. 实验环境

项目	内容
被测系统	Life Service 微服务系统
被测环境	阿里云 ECS 上的 Kubernetes/K3s 集群
网关入口	http://47.120.37.61:30081
压测工具	Apache JMeter 5.6.3
观测脚本	scripts/collect-hpa-experiment.sh
压测脚本	load-tests/run-hpa-jmeter.ps1
HPA 配置	k8s/hpa.yaml

3. HPA 配置摘要

服务	minReplicas	maxReplicas	CPU 目标
life-assistant-merchant-service	1	5	50%
life-assistant-api-gateway	1	3	60%
life-assistant-user-service	1	4	60%
life-assistant-order-service	1	4	60%
life-assistant-settlement-service	1	3	60%
life-assistant-fulfillment-service	1	4	60%

服务	minReplicas	maxReplicas	CPU 目标
life-assistant-engagement-service	1	4	60%

所有 HPA 均配置 `scaleDown.stabilizationWindowSeconds: 300`，压力停止后不会立即缩容。

4. 历史实验结果摘要

2026-09-01 的有效 HPA 实验记录显示：

指标	结果
总请求数	3711
成功请求数	3450
失败请求数	261
错误率	7.03%
吞吐量	6.07 req/s
平均响应时间	7290 ms
P95 响应时间	23737 ms
P99 响应时间	34899 ms

扩缩容观察：

- `merchant-service` 在压力升高后扩容至 5 个副本，压力下降后回落至 1。
- `api-gateway` 在压力升高后扩容至 3 个副本，压力下降后回落至 1。
- 两个服务均满足“升压扩容、降压缩容”的预期。

5. 问题分析

- HPA 行为符合预期，但 50 并发下系统出现明显延迟和 7.03% 错误率。
- `/api/search` 是主要瓶颈，原因是查询和组装商品时容易产生较多数据库访问和较大响应体。

- 单 Pod CPU 峰值接近 500m limit，说明资源限制、数据库连接池和查询逻辑都可能影响高负载表现。

6. 改进建议

- 优化 `/api/search`，避免按商家循环查询商品，改为批量查询并分组。
- 搜索结果分页，限制每个商家返回商品数量。
- 检查 `merchant.status`、`merchant.category`、`product.merchant_id`、`product.status` 等索引。
- 评估 Java 服务 CPU limit，从 500m 调整到 800m 或 1000m 后复测。
- 补充 20、30、40、50 并发梯度压测，定位稳定边界。

7. 交付说明

考虑到本地没有额外备份，2026-09-01 的 HPA/JMeter 原始实验材料已复原并继续纳入仓库：

- `reports/perf/hpa-20260901-102919/`
- `reports/perf/hpa-20260901-112703/`

当前交付同时保留 HPA 配置、实验步骤说明、本摘要报告、HTML 报告和 Kubernetes 观测 CSV/TXT。`.gitignore` 仅忽略后续本地重跑产生的 `*.jtl`、日志、压缩包和临时目录，避免新的噪声文件混入提交。

模型源文件附录

以下为随交付包提交的 Mermaid 模型源文件，可在支持 Mermaid 的编辑器中继续维护。

models/service-architecture.mmd

```
flowchart LR
    FE[React frontend] --> GW[API Gateway]
    GW --> US[user-service\nuser/address/cart/favorite]
    GW --> MS[merchant-service\nmerchant/product/search/dashboard]
    GW --> OS[order-service\norder/state/merchant-order]
    GW --> SS[settlement-service\ncoupon/payment]
    GW --> FS[fulfillment-service\nrider/delivery]
    GW --> ES[engagement-service\nreview/upload/message]
    US --> UDB[(user_db)]
    MS --> MDB[(merchant_db)]
    OS --> ODB[(order_db)]
    SS --> SDB[(settlement_db)]
    FS --> FDB[(fulfillment_db)]
    ES --> EDB[(engagement_db)]
    OS --> MS
    OS --> SS
    SS --> OS
    FS --> OS
    ES --> OS
    ES --> US
    ES --> MS
    US --> MS
```

models/cross-service-checkout.mmd

```
sequenceDiagram
    participant FE as React frontend
    participant GW as API Gateway
    participant OS as order-service
    participant US as user-service
    participant MS as merchant-service
    participant SS as settlement-service
    FE->>GW: POST /api/checkout
    GW->>OS: route checkout
    OS->>US: GET /internal/users/{userId}/addresses/{addressId}
    US-->>OS: address snapshot
    OS->>MS: POST /internal/products/quote
    MS-->>OS: product/spec price snapshot
    OS->>MS: POST /internal/products/reserve(requestId)
    MS-->>OS: reserved
    OS->>SS: POST /internal/coupon-locks
    SS-->>OS: locked discount
    OS->>OS: create orders/order_item/group_coupon
    OS->>US: DELETE /internal/users/{userId}/cart?merchantId=...
    OS-->>GW: order created
    GW-->>FE: Result<Order>
```

models/traceability.mmd

```
flowchart LR
    R[Requirements\nREQ01-REQ21] --> U[Use cases\nUC01-UC21]
    U --> F[Frontend pages\nLogin/Home/Checkout/Orders/Consoles]
    F --> G[API Gateway\n/api routes]
    G --> S[Microservices\nuser/merchant/order/settlement/fulfillment/engagement]
    S --> D[Database owners\nuser_db/merchant_db/order_db/settlement_db/fulfillment_db/engagement_db]
    S --> T[Test evidence\n84 microservice tests\n63 B-side tests\nfrontend E2E reports]
    S --> O[Operational evidence\nHPA/fault/Sentinel/performance reports]
    T --> C[Delivery checklist]
    O --> C
    C --> P[Final presentation focus\nUC07/UC08/UC11/UC14/UC16/UC18/UC20/UC21]
```